▸ **Aufgabe 1: Sprachen regulärer Ausdrücke** [3+3+3+3 Punkte]

Sei Σ = {A, B} ein Alphabet mit zwei Buchstaben. Betrachten Sie den folgenden regulären Ausdruck:

$$((A|B^*)A)^*$$

Entscheiden Sie zu jedem der folgenden Strings, ob er zur Sprache gehört, die von diesem regulären Ausdruck definiert wird.

1. Ist "ABC" in der von $((A|B^*)A)^*$ definierten Sprache?

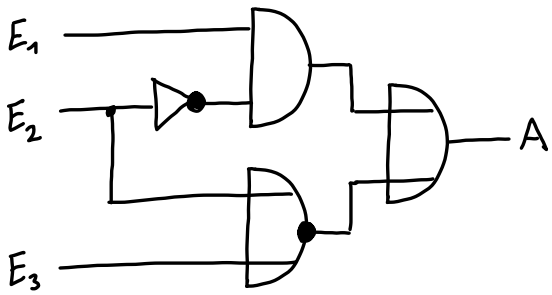
2. Ist "ABBA" in der von $((A|B^*)A)^*$ definierten Sprache?

3. Ist "AABABAA" in der von $((A|B^*)A)^*$ definierten Sprache?

4. Ist der Leerstring ε = "" in der von $((A|B^*)A)^*$ definierten Sprache?

▸ **Aufgabe 2: Wahrheitstabellen logischer Schaltungen** [14 Punkte]

Betrachten Sie folgende logische Schaltung mit drei Eingängen E_1, E_2 und E_3 und einem Ausgang A:



Die folgende Tabelle soll die Wahrheitstabelle für diese Schaltung werden:

E_1	E_2	E_3	A
0	0	0	
0	0	1	
0	1	0	
0	1	1	
1	0	0	
1	0	1	
1	1	0	
1	1	1	

Füllen Sie alle Felder in der Spalte A aus, so dass sie der Ausgabe der Schaltung entspricht.

▸ **Aufgabe 3: Zweierkomplement** [12 Punkte]

Die Zahl 50 ("fünfzig") wird als 8-bit Dualzahl mit 00110010 kodiert. Wie wird die Zahl −50 ("minus fünfzig") als 8-bit Dualzahl (*signed integer*) kodiert?

▸ Aufgabe 4: Disjunktive Normalform [14 Punkte]

Betrachten Sie folgende Wahrheitstabelle für eine Ausgabe A in Abhängigkeit von drei Variablen E_1, E_2 und E_3:

E_1	E_2	E_3	A
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	0

Geben Sie für die logische Funktion A, die durch diese Wahrheitstabelle definiert ist, einen *logischen Ausdruck in Disjunktiver Normalform* (DNF) an.

Anmerkung: In dieser Aufgabe sollen Sie keinen Schaltkeis zeichnen, sondern einen logischen Ausdruck hinschreiben!

▸ Aufgabe 5: Wer A sagt darf nicht B sagen [14 Punkte]

Sei $\Sigma = \{A, B, C\}$ ein Alphabet mit drei Buchstaben, Σ^* die Menge aller Strings aus Buchstaben aus Σ und $L \subseteq \Sigma^*$ eine Teilmenge von Σ^*, nämlich:

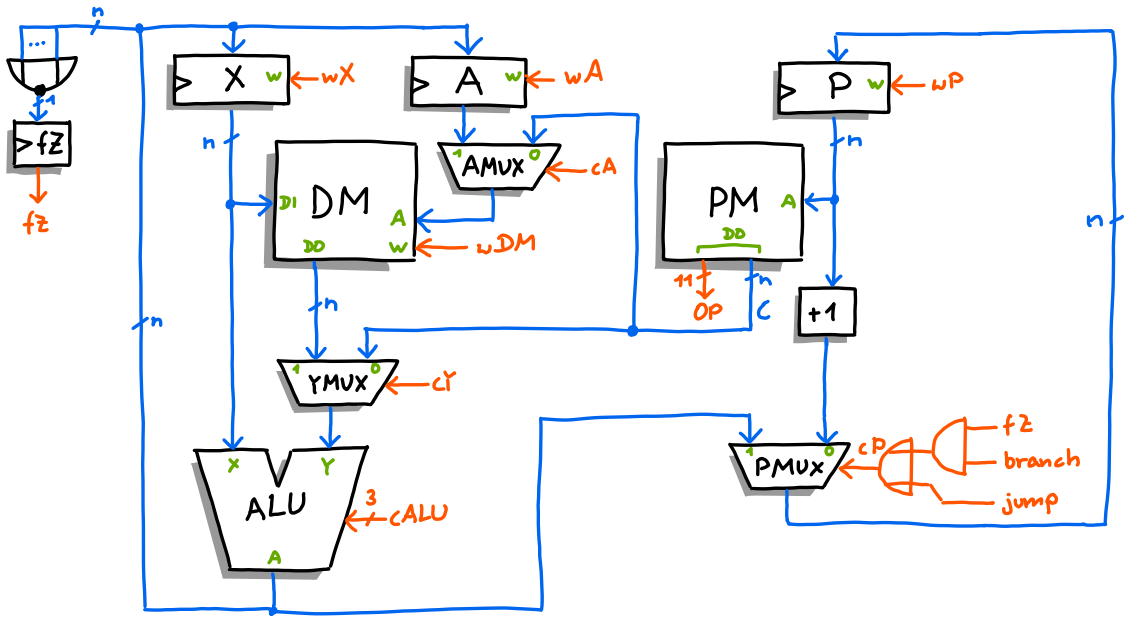
$L = \{w \in \Sigma^* \mid w$ enthält "AB" *nicht* als Teilstring$\}$

Beispiele für Strings in L: "A", "BBA", "BACB", "CC" und das leere Wort ε.
Beispiele für Strings, die nicht in L sind: "AB", "BABC" und "BBAB"

Zeichnen Sie einen *deterministischen endlichen Automaten* (DEA), der genau die Strings aus L akzeptiert. Ihr Automat muss also alle Strings aus L akzeptieren, darf aber andererseits keinen String akzeptieren, der nicht in L ist. Kennzeichnen Sie in Ihrem Automaten den Startzustand und alle akzeptierenden Endzustände, und beschriften Sie alle Kanten mit Buchstaben aus Σ.

▸ Aufgabe 6: The Big Coffee Machine [8+12 Punkte]

Zur Erinnerung sehen Sie hier die Schaltung der BCM (Version 3) sowie Listen zur Programmierung der BCM in Assembler und eine Tabelle zur Bedeutung unterschiedlicher Steuersignale cALU:



Laden und Speichern

Mnemonic	Effekt
LDXC	X ← c
LDX	X ← DM[c]
LDXA	X ← DM[A]
STX	DM[c] ← X
STXA	DM[A] ← X
XADC	A ← X + c
LDAC	A ← c
LDA	A ← DM[c]

Rechenoperationen

Mnemonic	Effekt
ADD	X ← X + DM[c]
SUB	X ← X − DM[c]
CMP	fZ ← 1 wenn X = DM[c]
AND	X ← X *and* DM[c]
OR	X ← X *or* DM[c]
NOT	X ← *not* X
LSH	X ← X << DM[c]
RSH	X ← X >> DM[c]

zu jeder Rechenoperation (außer NOT) gibt es eine C-Variante (z.B. ADDC: X ← X + c) und eine A-Variante (z.B. ADDA: X ← X + DM[A])

cALU Kodierungen

cALU	Ausgang A	
000	Y	
001	X + Y	
010	X − Y	
011	X *and* Y	} bitweise
100	X *or* Y	
101	*not* X	
110	X << Y	
111	X >> Y	

Sprünge und Sonstiges

Mnemonic	Effekt
JMP	P ← c
BZ	P ← c, wenn fZ = 1
LDP	P ← DM[c]
END	(Programm endet)

(Fortsetzung von Aufgabe 6)

1. In der folgenden Tabelle sind die Steuersignale für drei Operationen der BCM aufgeführt. Jede Zeile steht für eine Operation. Füllen Sie die Tabelle komplett aus.

Effekte	wP	wX	wA	wDM	cALU	cY	cA
$X \leftarrow c$	1	1	0	0	000	0	0
$A \leftarrow X + c$							
$DM[A] \leftarrow X$ und $X \leftarrow c$							

2. Schreiben Sie ein Assembler-Programm auf, das in die Speicherzelle DM[8] den Wert aus der Speicherzelle DM[2] schreibt, wenn in DM[2] der gleiche Wert wie in DM[3] gespeichert ist; andernfalls soll das Programm die Zahl 4 nach DM[8] schreibt. Wenn die BCM Ihr Programm durchlaufen hat, soll also gelten:

$$DM[8] = \begin{cases} DM[2], \text{ wenn } DM[2] = DM[3] \\ 4 \quad\quad , \text{ wenn } DM[2] \neq DM[3] \end{cases}$$

▸ **Aufgabe 7: Schaltung zur Parität** [4+4+4+4 Punkte]

Sei $n \in \mathbb{N}$ eine natürliche Zahl. Die Schaltung n-Parity habe n Eingänge E_1, E_2, …, E_n und einen einzelnen Ausgang A. Der Wert am Ausgang hänge dabei von der Anzahl der Einsen ab, die an den Eingängen anliegen: Ist die Zahl der Einsen an den Eingängen gerade, so soll der Ausgang A = 0 sein, ansonsten soll A = 1 sein. Die Wahrheitstabelle für 3-Parity würde zum Beispiel so aussehen:

E_1	E_2	E_3	Anzahl Einsen	A
0	0	0	0	1
1	0	0	1	0
0	1	0	1	0
1	1	0	2	1
0	0	1	1	0
1	0	1	2	1
0	1	1	2	1
1	1	1	3	0

Hinweis: Es geht in dieser Aufgabe nicht darum, ob die Eingänge eine gerade oder ungerade Dualzahl kodieren würden, sondern wie viele Eingänge 1 bzw. 0 sind.

1. Zeichnen Sie eine Schaltung für 1-Parity.

2. Zeichnen Sie eine Schaltung für 2-Parity.

Anmerkung: Für Ihre Schaltungen dürfen Sie ausschließlich Gatter vom Typ *and*, *or*, *nand* und *nor* − jeweils mit zwei oder auch mehr Eingängen − sowie *not*-Gatter verwenden. Im nächsten Aufgabenteil 3 dürfen Sie darüber hinaus auch k-Parity (aber nicht 2k-Parity!) als Baustein verwenden.

(Fortsetzung von Aufgabe 7)

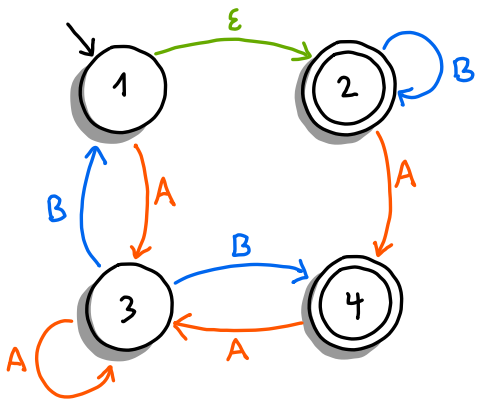
3. Durch geschicktes zusammenschalten von zwei k-Parity-Schaltungen (für ein festes $k \in \mathbb{N}$) mit einigen weiteren Gattern kann man eine Schaltung für 2k-Parity (also eine Schaltung mit $n = 2 \cdot k$ Eingängen) bauen. Zeichnen Sie eine Schaltung für 2k-Parity, wobei Sie k-Parity als Baustein verwenden dürfen.

4. Aus den Aufgabenteilen 1 bis 3 ergibt sich ein rekursives Schema, mit dem man für beliebige Zweierpotenzen n eine Schaltung für n-Parity aufbauen kann. Welche Kosten und Tiefen haben die auf diese Weise erzeugten Schaltungen? Füllen Sie die folgende Tabelle mit den Kosten und Tiefen für verschiedene $n \in \mathbb{N}$ aus.

n	Kosten von n-Parity	Tiefe von n-Parity
1		
2		
4		
8		
16		

▸ **Aufgabe 8: ε-NEA in DEA umwandeln** [16 Punkte]

Sei Σ = {A, B} ein Alphabet mit zwei Buchstaben. Betrachten Sie folgenden nicht-deterministischen endlichen Automaten (NEA) mit ε-Übergängen:



Der ε-NEA hat also vier Zustände: Zustand 1 ist der Startzustand und die Zustände 2 und 4 sind akzeptierende Endzustände.

Verwandeln Sie diesen ε-NEA in einen *deterministischen* endlichen Automaten (DEA). Kennzeichnen Sie in Ihrer Zeichnung den Startzustand und alle akzeptierenden Endzustände des DEA. Beschriften Sie außerdem jeden Zustand Ihres DEA mit der ihm entsprechenden (Teil-) Menge erreichbarer Zustände des ε-NEA. Ihr DEA darf nicht mehr als $2^4 = 16$ Zustände haben.

(Blatt für Notizen)